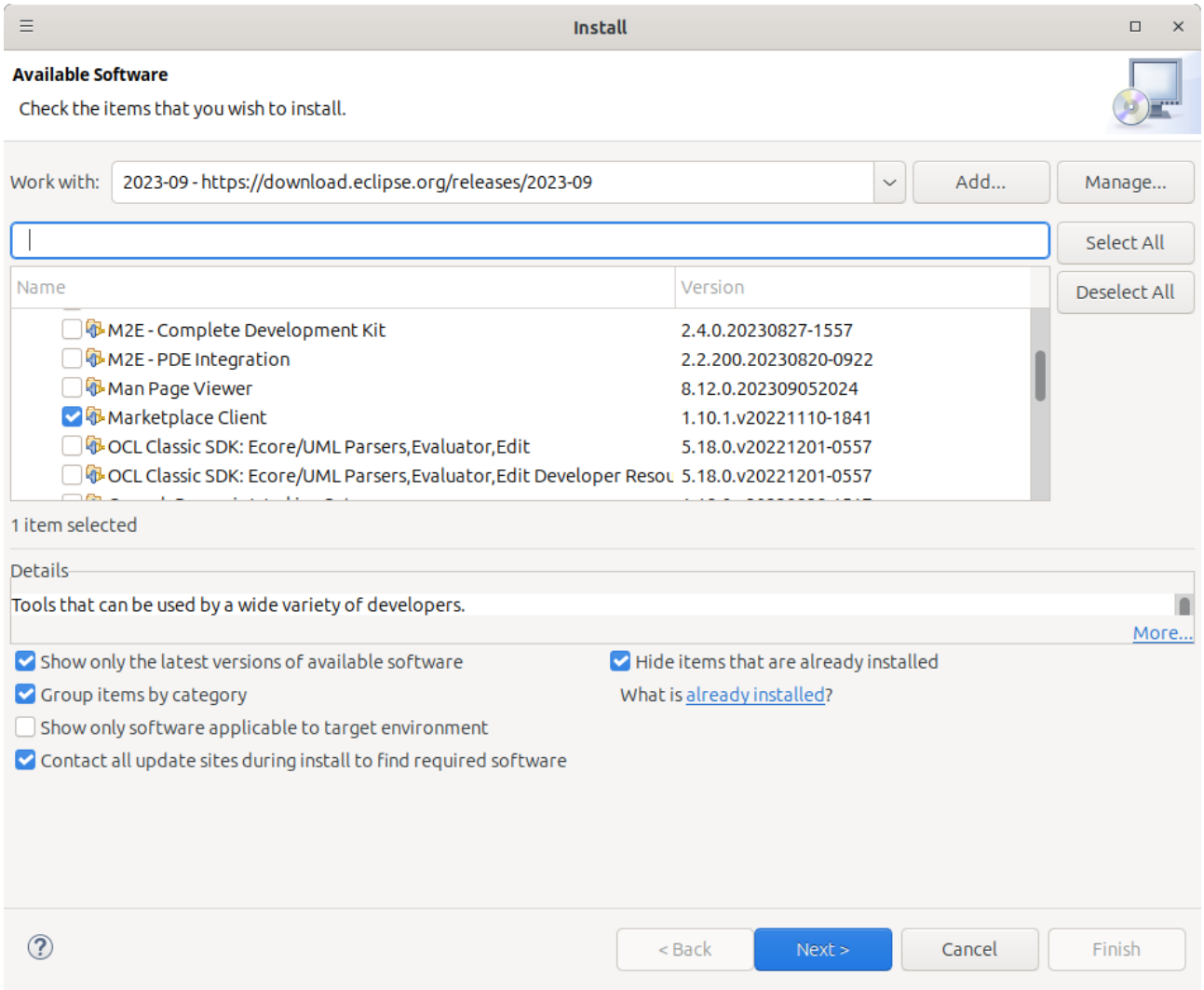
En este documento se describe la configuración inicial de Eclipse para trabajar con Hibernate. Las versiones usadas para ello son las siguientes:

- **ECLIPSE 2023-09**
- **Hibernate 6.4.1**
- **Conector J (antes llamado JDBC): 8.2.0**
- **Jboss Tools 4.29.0**

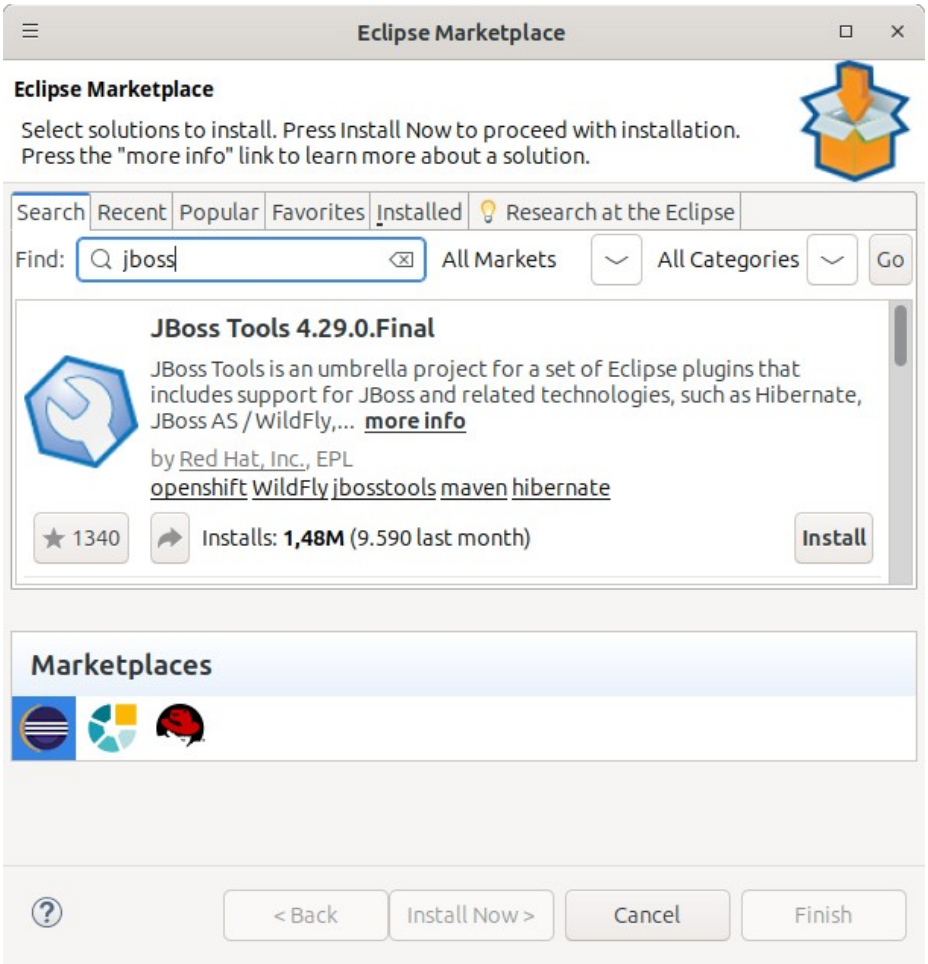
Actualizamos el software de Eclipse (en **Help→ Check for Updates**)

Vamos a **Help→Eclipse Marketplace**

NOTA: Si no lo tenemos entramos en Help→Install New Software e instalamos antes el Marketplace Client



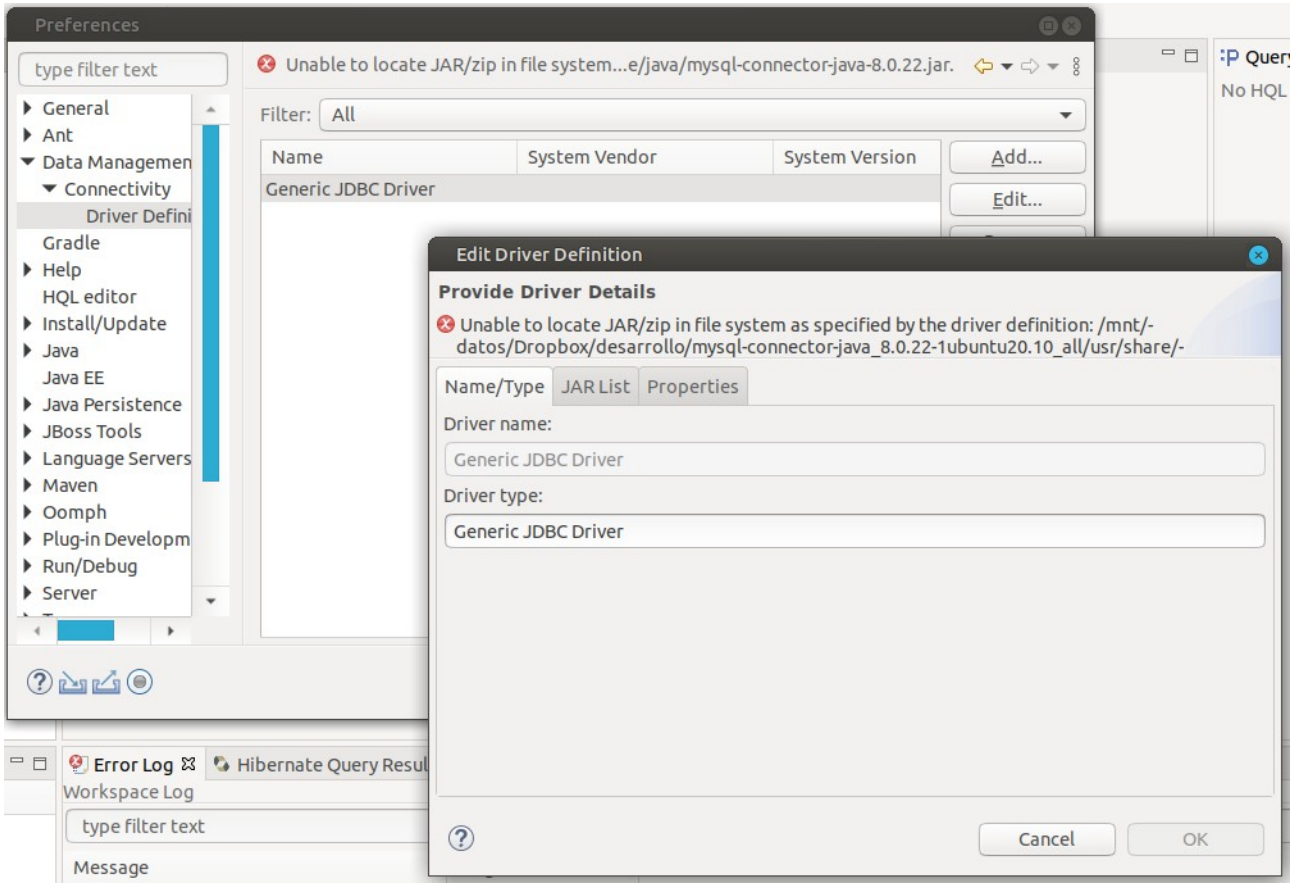
Una vez que tenemos disponible el marketplace buscamos **Jboss Tools** y lo instalamos con las opciones recomendadas:



Entramos ahora en:

Windows-> Preferences -> Data Management -> Connectivity -> Driver Definition

y configuramos el **Generic JDBC Driver**

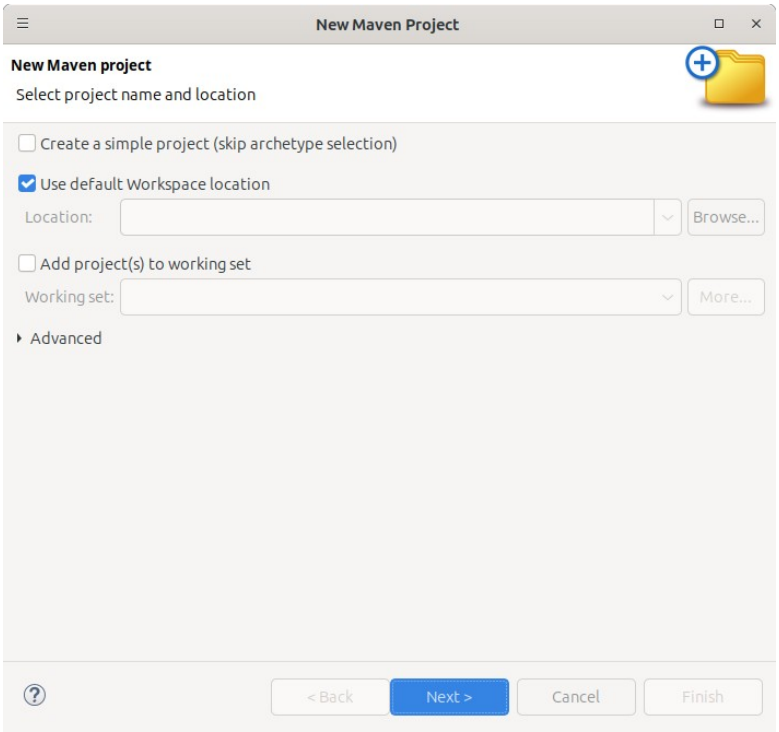


Los campos a rellenar son:

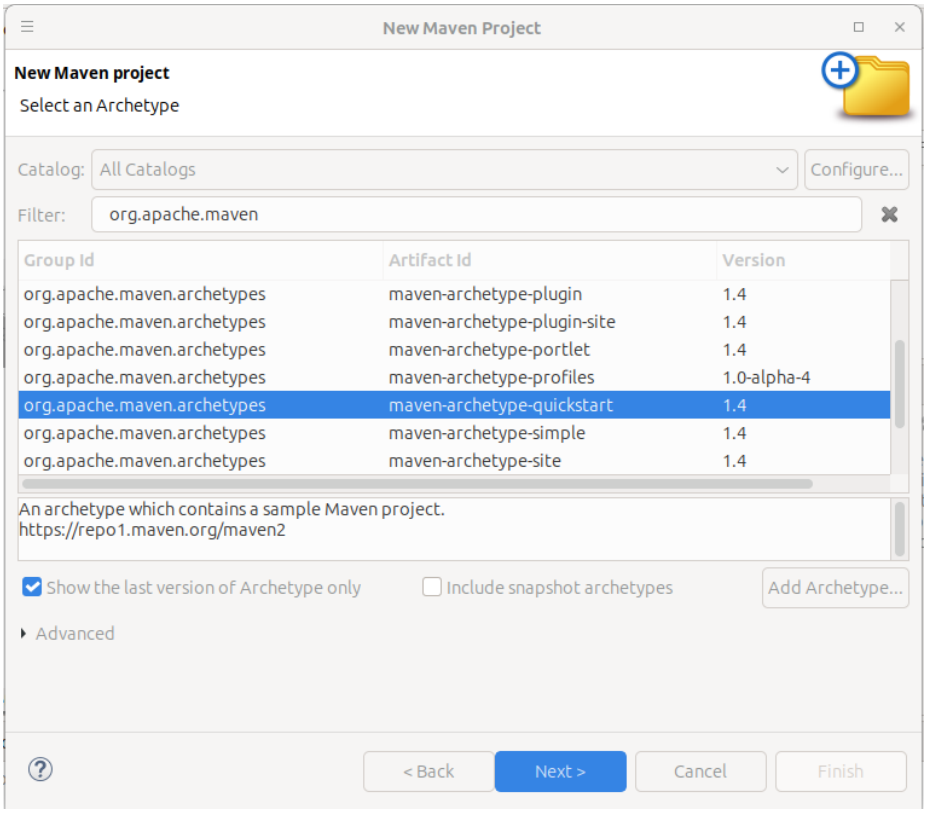
- **Name/Type: Generic JDBC Driver**
- **Jar List: Añadir el .jar descargado con el conector J**
- **Properties - Driver Class -> com.mysql.cj.jdbc.Driver**

IMPORTANTE: Hasta aquí hemos hecho una configuración genérica. No hay que volver a hacerlo salvo que reinstalemos eclipse.
Lo que viene a partir de aquí es la configuración que hay que hacer en cada proyecto en el que vamos a usar Hibernate.

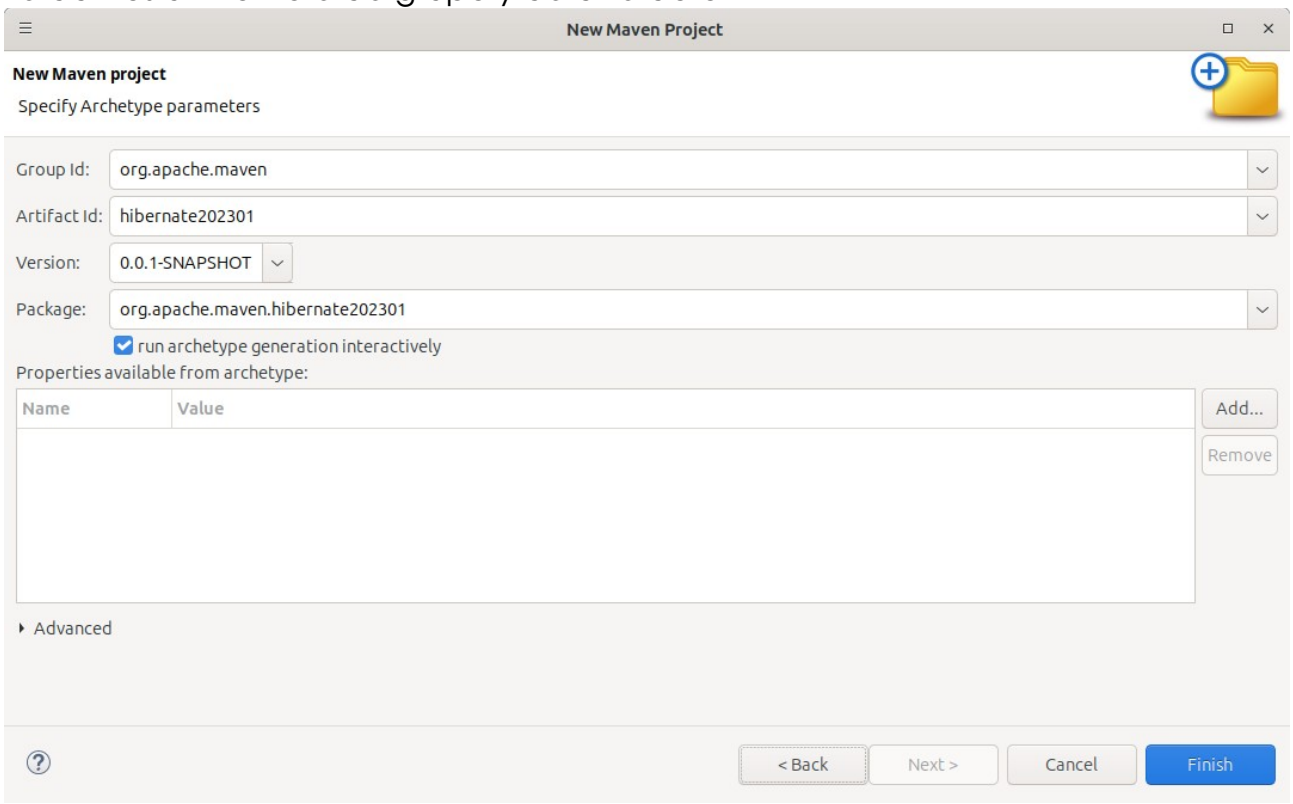
Para crear un proyecto con Hibernate partimos de un **Proyecto Maven**



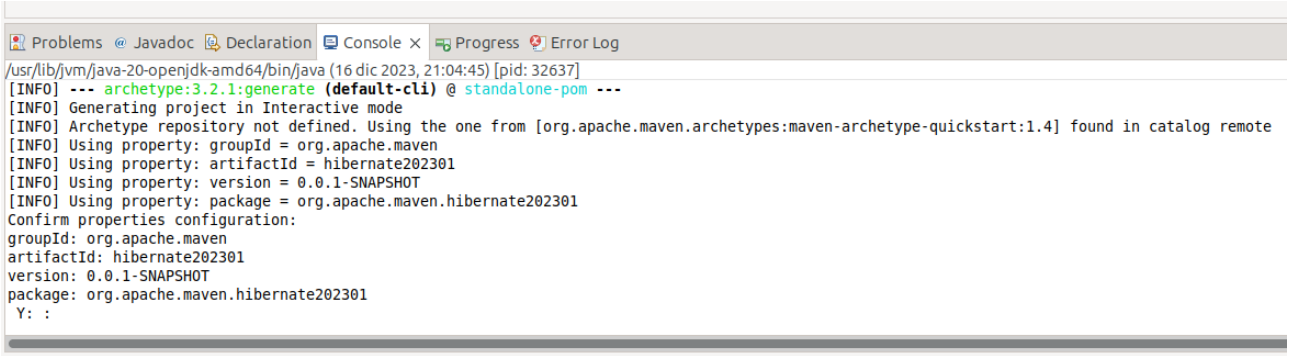
Escogemos como plantilla **maven-archetype-quickstart**



Le damos un nombre de grupo y de artefacto:



Durante la creación del proyecto se nos pedirá confirmación antes de finalizar en la consola de Eclipse:



Una vez creado el fichero, si lo desplegamos veremos un fichero que se llama **pom.xml.** Ahí tenemos que incluir las dependencias del conector J y de Hibernate.

Podemos copiarlas de la página **mvnrepository.com**. En concreto de las siguientes páginas:

https://mvnrepository.com/artifact/com.mysql/mysql-connector-j/
https://mvnrepository.com/artifact/org.hibernate.orm/hibernate-core

Seleccionamos la versión que vamos a usar y copiamos el texto que nos aparece en cada una. A Continuación tienes las de las versiones 8.2.0 y 6.4.1 que son las que vamos a usar nosotros:

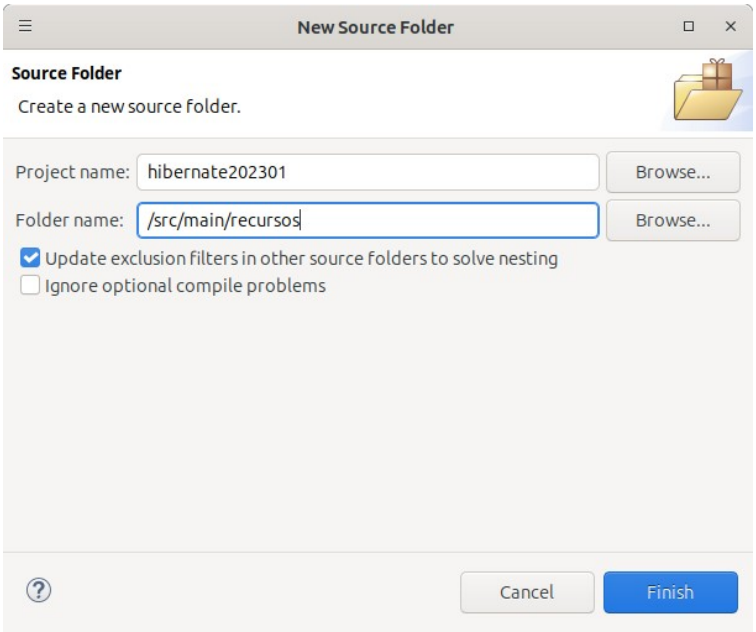
```
<!-- https://mvnrepository.com/artifact/org.hibernate.orm/hibernate-core -->
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.1.Final</version>
</dependency>

<!-- https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.2.0</version>
</dependency>
```

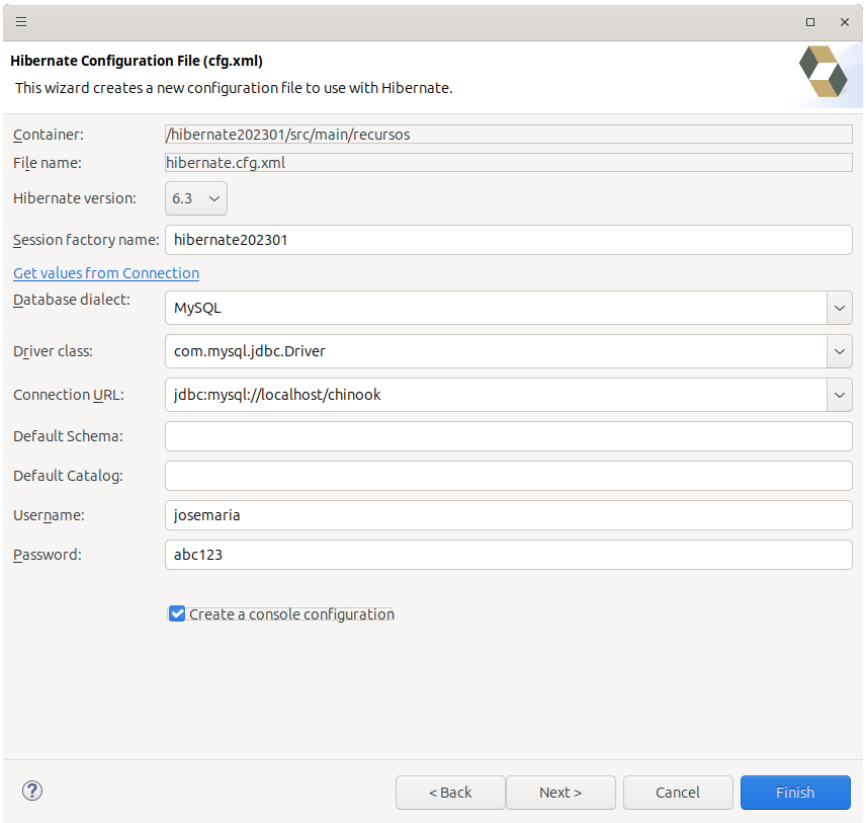
La sección de dependencias del fichero pom.xml debería de quedarnos así:

```
20
21  <dependencies>
22    <dependency>
23      <groupId>junit</groupId>
24      <artifactId>junit</artifactId>
25      <version>4.11</version>
26      <scope>test</scope>
27    </dependency>
28    <!-- https://mvnrepository.com/artifact/org.hibernate.orm/hibernate-core -->
29    <dependency>
30        <groupId>org.hibernate.orm</groupId>
31        <artifactId>hibernate-core</artifactId>
32        <version>6.4.1.Final</version>
33    </dependency>
34    <!-- https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
35    <dependency>
36        <groupId>com.mysql</groupId>
37        <artifactId>mysql-connector-j</artifactId>
38        <version>8.2.0</version>
39    </dependency>
40  </dependencies>
41
```

Me creo ahora una carpeta nueva de tipo Source Folder para guardar la configuración de hibernate. Marca la opción de **Update exclusion filters.** Yo la he llamado recursos:



Para empezar a configurar Hibernate en nuestro proyecto pulsamos en **New -> Other -> Hibernate -> Hibernate Configuration File. c**rearemos el fichero en la carpeta /src/main/recursos que hemos creado:



Fíjate en los datos que hemos rellenado:

- **Seccion factory name**
- **Database Dialect**
- **Driver class**
- **Connection URL**
- **Username**
- **Password**
- **Create a console configuration (márcalo!)**

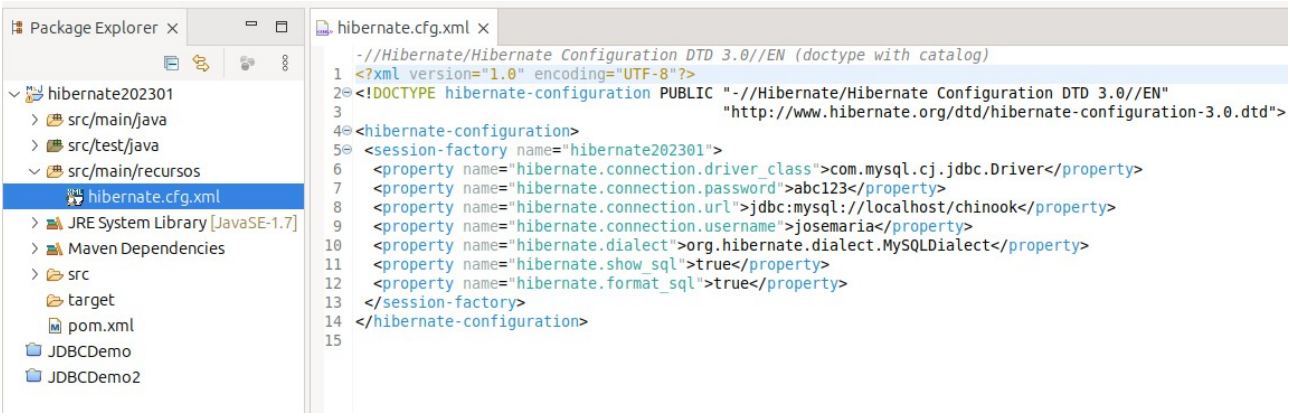
IMPORTANTE: Los datos que damos aquí son los de conexión con la base de datos que usará nuestro programa. Aquí estoy usando un usuario con permisos sobre todas las bases de datos del sistema. Sería mucho mas correcto utilizar un usuario que sólo tenga acceso sobre la base de datos sobre la que vamos a trabajar. Y en un proyecto en producción es impresindible que sea así por motivos de seguridad!

Una vez salvada la configuración tenemos que editar manualmente en la ficha de propiedades lo siguiente:

Driver class: com.mysql.cj.jdbc.Driver

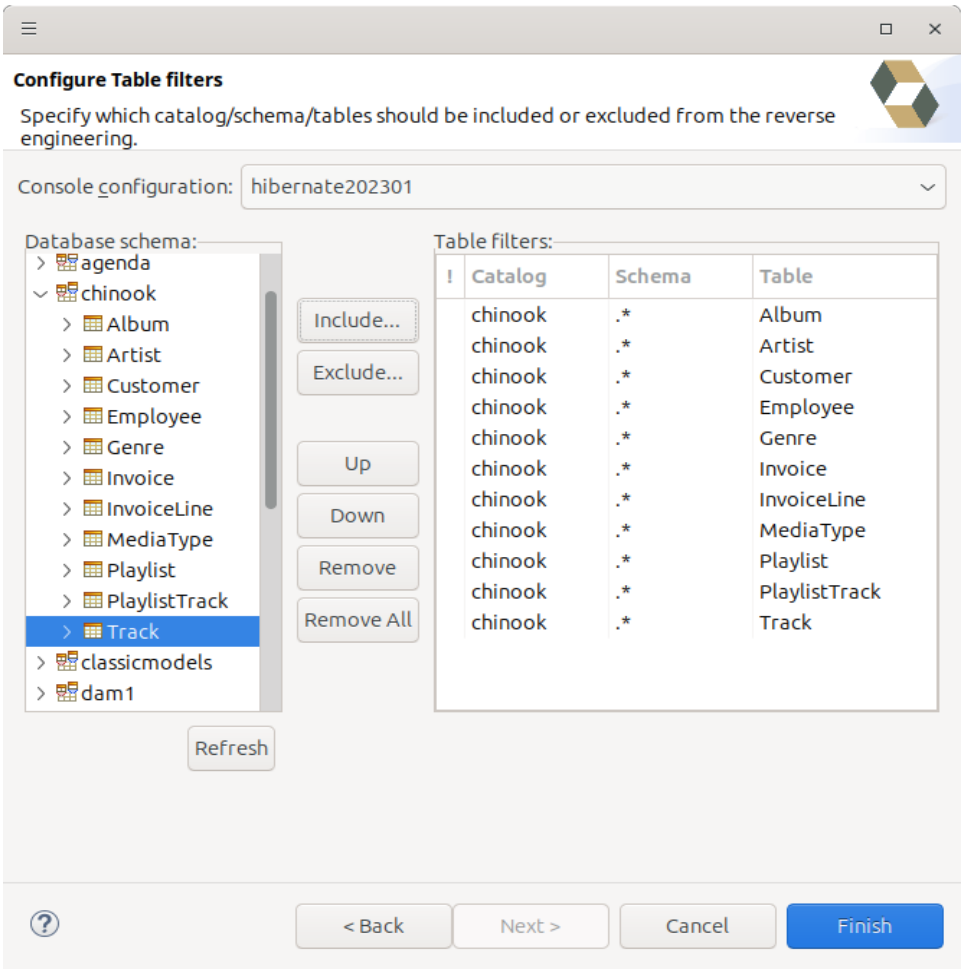
Este cambio lo hacemos porque el que hemos escogido en el asistente está desaconsejado, pero no aparece aún el que hay que usar ahora

Podemos editar a mano el fichero de configuración en lugar de con el asistente si así lo preferimos:



Ahora creamos el fichero **XML Hibernate Reverse Engineering** eligiendo **New→Other→Hibernate** en el mismo directorio de recursos que creamos antes

NOTA: la base de datos debe de existir y tener creada alguna tabla de ejemplo previamente para que podamos generar las clases.



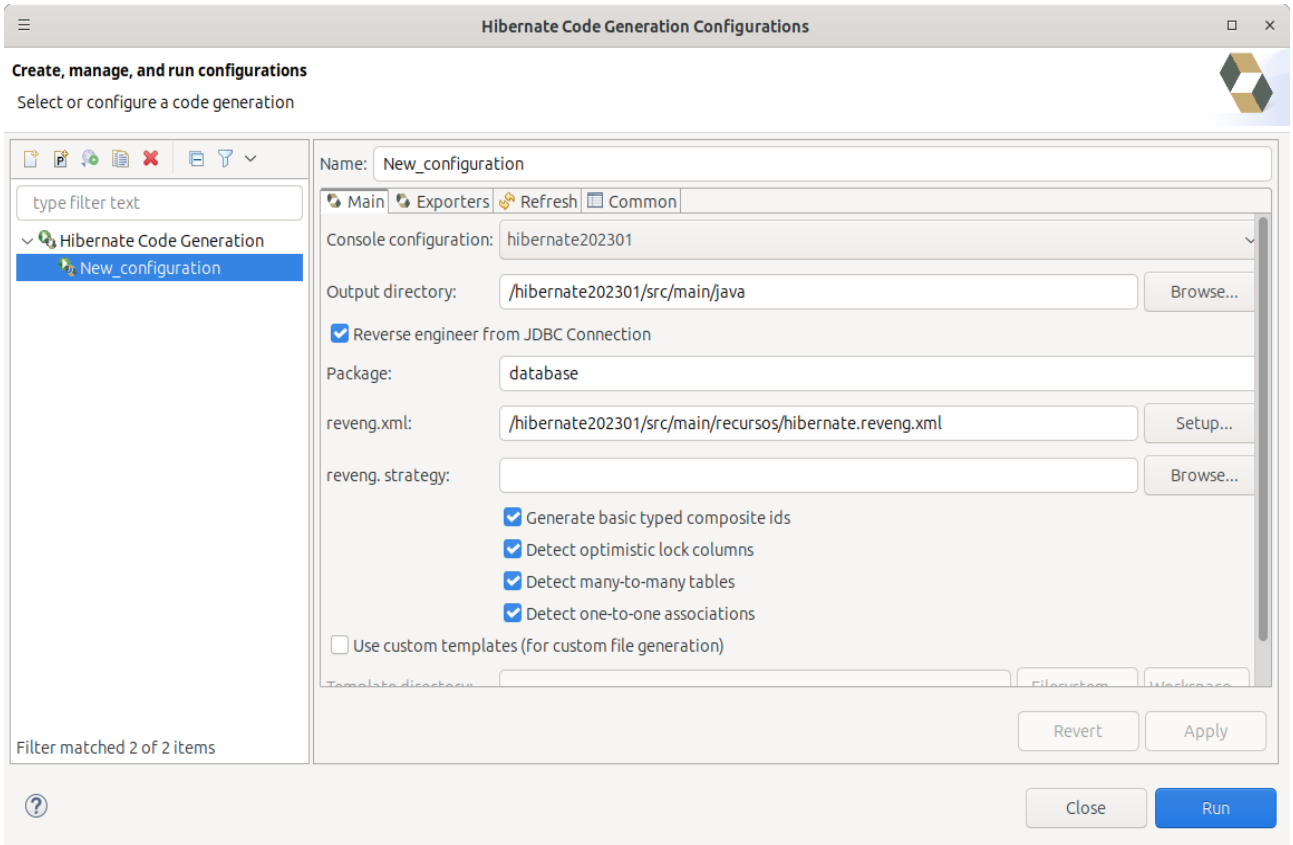
Elegimos correctamente el nombre de nuestro **Console Configuration** (como lo ha creado de forma automática tendrá el mismo nombre que el proyecto) y pulsamos Refresh. A la izquierda debería de aparecer la base de datos y las tablas que tenga. Seleccionamos las tablas (¡individualmente, cuidado!) con las que vamos a trabajar y pulsamos include

Aquí vemos que como el usuario que hemos elegido tiene permisos sobre todas las bases de datos nos aparecen todas. Si hubiéramos utilizado un usuario que sólo tuviera acceso a chinook sería la única que veríamos.

Para generar las clases entramos en

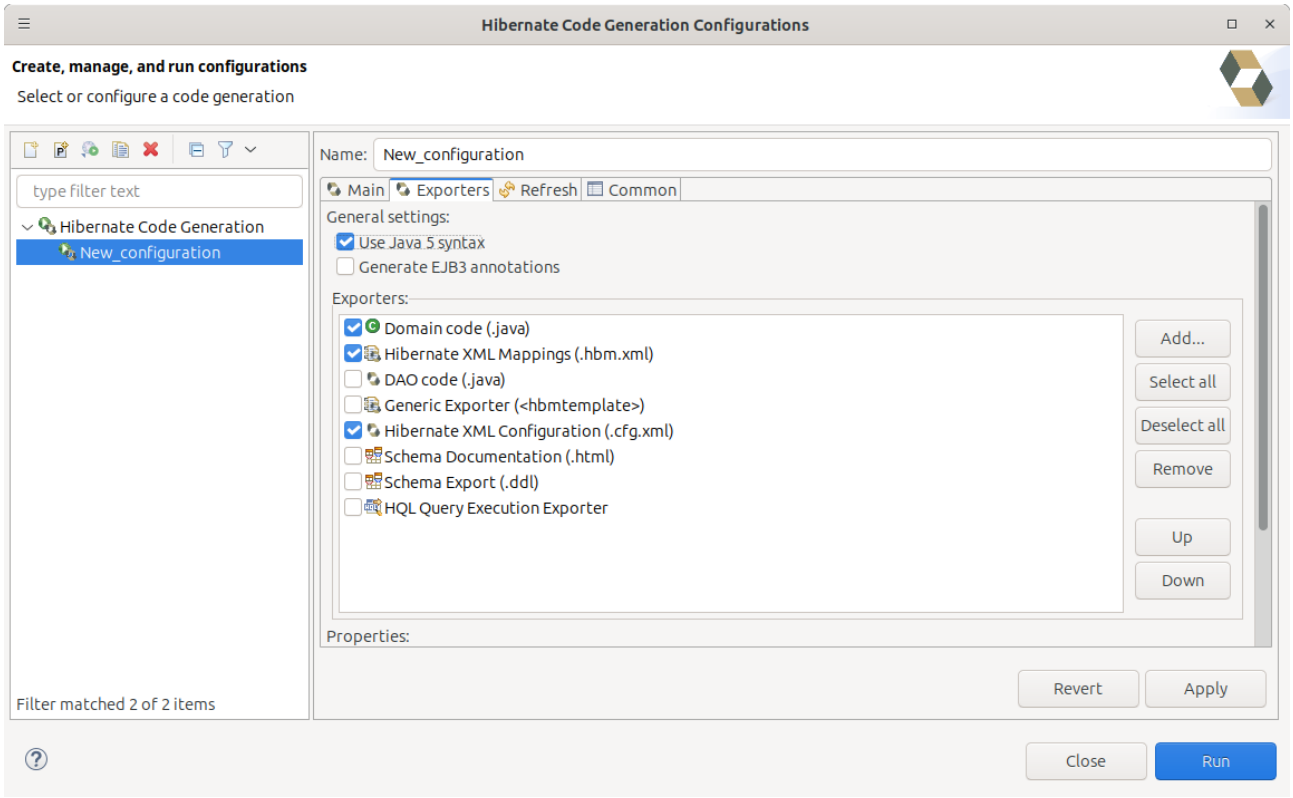
Run -> Hibernate Code Generation -> Hibernate Code Generation Configurations

Hacemos click en **Hibernate Code Generation**. Elegimos el fichero de **console configuration** y seleccionamos el **Output directory** (¡pon el mismo que aparece ahí!), le ponemos nombre al package y seleccionamos el fichero **reveng.xml**



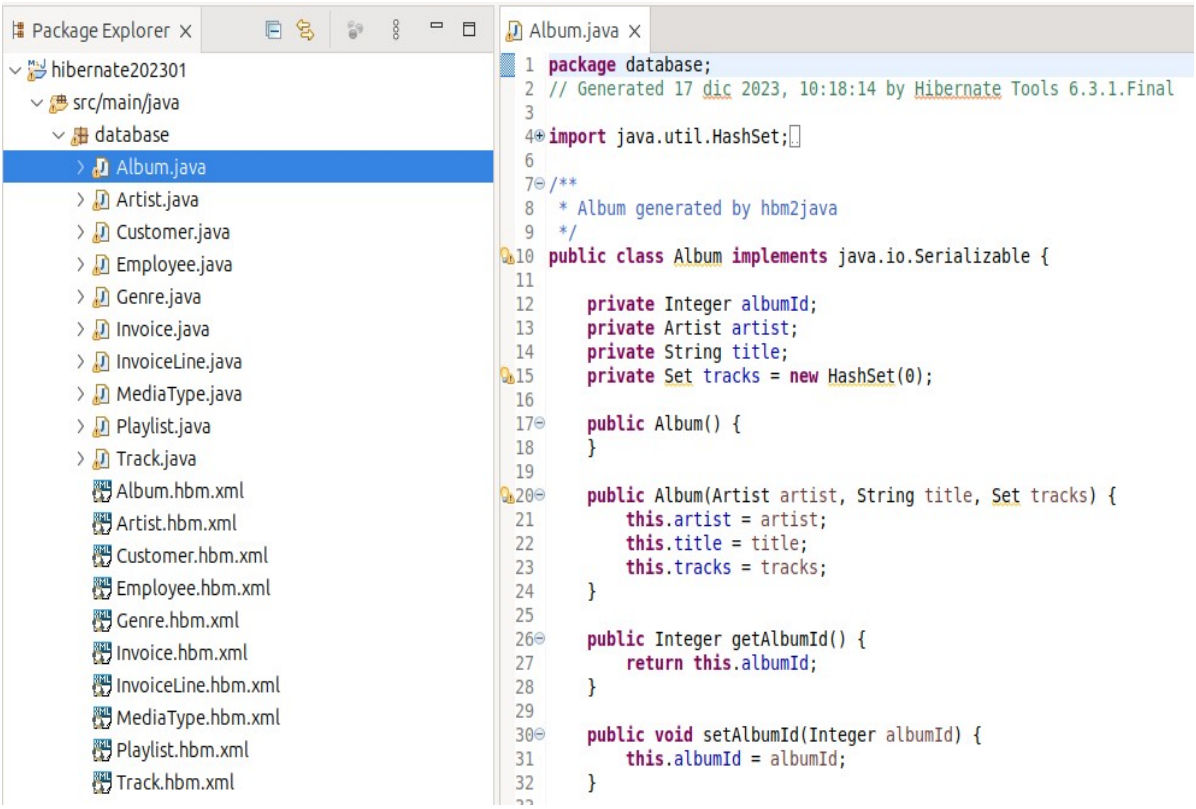
NOTA: Tenemos que tener especial cuidado en esta ventana porque conserva las elecciones que hemos hecho en proyectos anteriores. Revisad cuidadosamente cada uno de los campos!!!

En la pestaña **Exporters** marcamos:

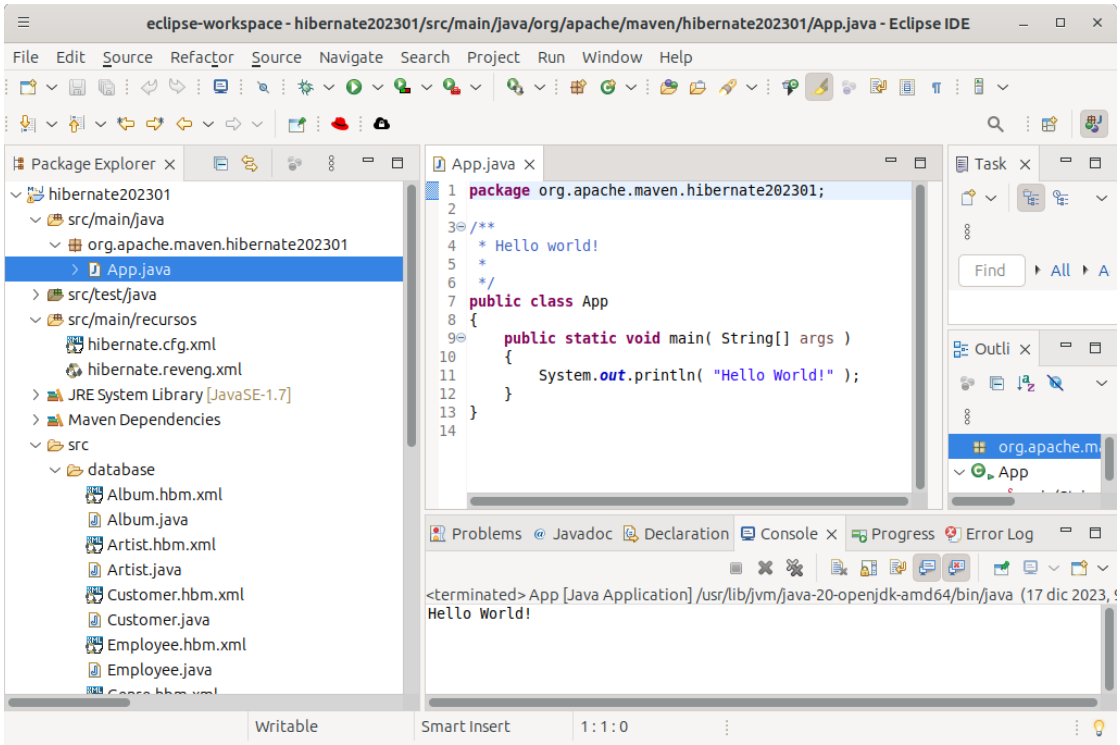


Pulsamos Apply y Run

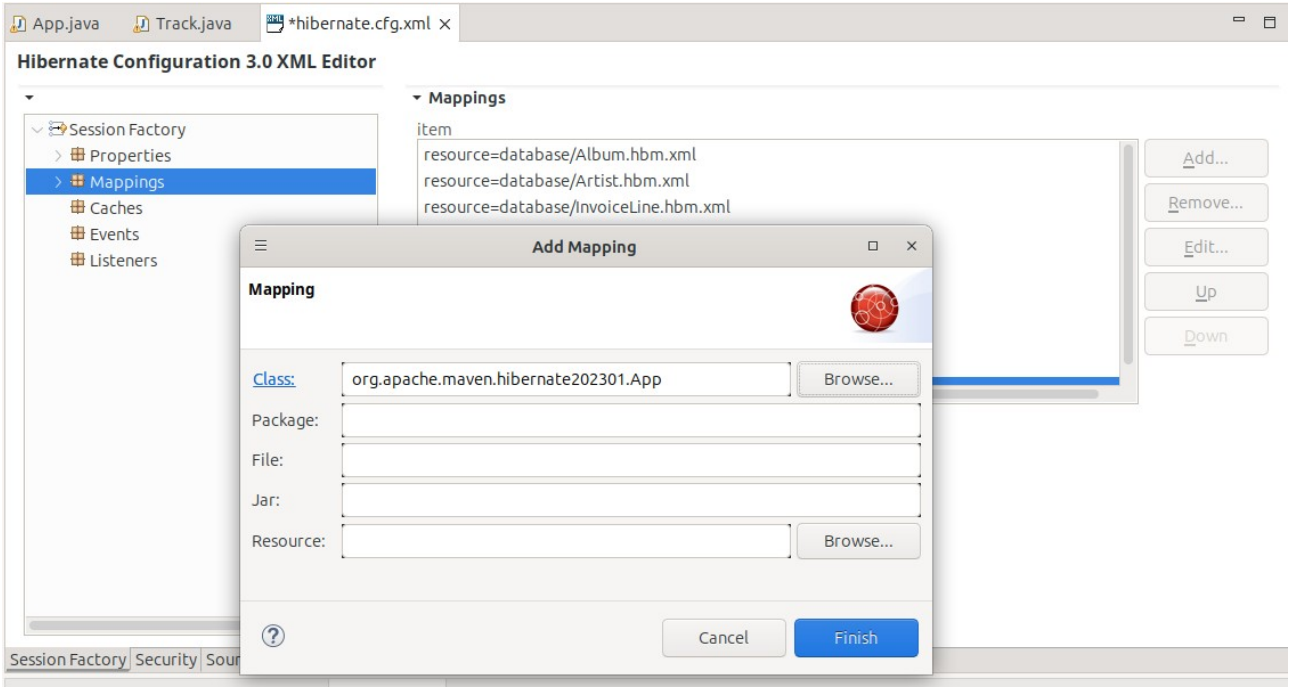
Nos debería de generar un paquete con las clases de cada tabla y los getters y setters correspondientes



Vamos ahora a crear ya nuestra aplicación. Vemos que con la plantilla del proyecto ya nos ha creado un Hello World. Lo abrimos y lo ejecutamos para probarlo:



Si vamos a usar esa clase (App) para nuestra aplicación tenemos que mapearla en el fichero de configuración de Hibernate:



Luego sustituimos el código de la clase por el de esta plantilla que será la que usaremos como base en nuestros proyectos:

```
package org.apache.maven.hibernate202301;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;
import java.util.logging.Level;
import java.util.logging.Logger;

import database.*;

public class App {

    private static Session sesion;
    private static Transaction tx;

    public static void main( String[] args ) {
      // La siguiente línea elimina los mensajes informativos en consola
      Logger.getLogger("org.hibernate").setLevel(Level.SEVERE);
```

```
        SessionFactory singleton = new
Configuration().configure().buildSessionFactory();

        sesion = singleton.openSession();

        tx =sesion.getTransaction();
        tx.begin();

        try {
            // Aquí nuestro código...

            tx.commit(); // si nos hace falta...
        }catch (HibernateException e) {
            e.printStackTrace();
        } finally {
            sesion.close();
            singleton.close();
        }
    }
}
```

NOTA: Por si no lo conoces, un singleton (instancia única) es un patrón de diseño muy usado en proyectos Java. Tienes mas información acerca de su funcionalidad y para que se usa aquí:

https://es.wikipedia.org/wiki/Singleton

Y aquí tienes un primer ejemplo de un método que puedes usar con la clase anterior para hacer una consulta a la base de datos chinook:

```
    public static void buscarTema(String nom) {
      Query q = sesion.createQuery("from Track where name = :nombre",
Track.class);
            q.setParameter("nombre", nom);
            List listaTemas = q.list();
            if (listaTemas.isEmpty())
                  System.err.println("No existe ningún tema con ese nombre");
            else {
                  for (int i = 0; i < listaTemas.size(); i++) {
                        Track tema = (Track)listaTemas.get(i);
                        System.out.println(tema.getName());
                        Album album = tema.getAlbum();
                        System.out.println("   - Álbum: " + album.getTitle());
                        Artist artista = album.getArtist();
                        System.out.println("   - Artista: " +
artista.getName());
                  }
            }
    }
```